Information and Software Technology 198 (2026) 108208

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

Review article

A review on Python libraries for temporal network analysis

Claudio D.G. Linhares [a,*], Jean R. Ponciano [b], Martim R. Oliveira [a], Amilcar Soares [a], Agma J.M. Traina [b], Andreas Kerren [c,a]

[a] Department of Computer Science, Linnaeus University, Växjö, Sweden
[b] Institute of Mathematics and Computer Sciences, University of São Paulo, São Carlos, São Paulo, Brazil
[c] Department of Science and Technology, Linköping University, Norrköping, Sweden

ARTICLE INFO

Keywords:
Temporal networks
Python
Network metrics
Libraries
Packages

ABSTRACT

Context: Complex networks represent systems with non-trivial connections and are widely used in fields such as social media, biology, and transportation. Temporal networks extend this by capturing the evolution of connections over time, providing insights into event sequences and information diffusion. Analyzing these networks requires specialized tools, and Python offers a variety of libraries tailored for this purpose.
Objective: This study evaluates Python libraries designed for temporal network analysis based on multiple criteria. The aim is to assess the strengths and limitations of these tools, guide users in selecting appropriate libraries, and identify gaps for future development.
Methods: A comparative analysis was conducted on selected Python libraries using predefined evaluation criteria. The assessment considered factors such as available documentation, supported metrics, visualization capabilities, supported format, uniqueness, community support, and popularity. Data were gathered from official documentation, community forums, scientific papers, and usage statistics.
Results: Findings indicate that the TGX, Teneto, and PathpyG stand out, excelling in three of five criteria. Networkx-t shows balanced performance with no significant drawbacks, making it a reliable general-purpose choice. However, several tools have limitations in specific areas, such as a lack of comprehensive documentation or advanced visualization features.
Conclusion: This review provides an overview of existing Python tools for temporal network analysis, offering insights into their capabilities and shortcomings. The results assist researchers and practitioners in selecting suitable libraries while highlighting areas for improvement and potential future developments in the field.

Contents

* Corresponding author.
E-mail addresses: claudio.linhares@lnu.se (C.D.G. Linhares), jeanponciano@usp.br (J.R. Ponciano), martim.olv@gmail.com (M.R. Oliveira), amilcar.soares@lnu.se (A. Soares), agma@icmc.usp.br (A.J.M. Traina), andreas.kerren@lnu.se, andreas.kerren@liu.se (A. Kerren).

https://doi.org/10.1016/j.infsof.2026.108208
Received 15 March 2025; Received in revised form 5 April 2026; Accepted 27 May 2026
Available online 1 June 2026
0950-5849/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

1. Introduction

Complex networks, also referred to as graphs, are systems composed of interconnected elements or nodes, where the relationships between these elements exhibit certain structural patterns or characteristics rather than being purely random [1]. These networks are prevalent across various domains, including social networks (e.g., friendships on social media), biological networks (e.g., protein relationships), transportation networks (e.g., airline flights), and many others [2]. Understanding the structure and dynamics of these networks is essential for uncovering unexpected behaviors and patterns in the data and for creating robust, efficient systems [3].

Temporal (or dynamic) networks add a dimension to this analysis by incorporating the evolution and distribution of relationships over time [2]. This dynamic perspective allows researchers to gain insights into the temporal ordering of events, the propagation of information, and the evolution of structures, offering more accurate modeling and prediction of system behavior across diverse fields [3]. The ability to analyze temporal patterns is crucial for understanding evolving systems and developing adaptive strategies that can respond to changes, thereby enhancing robustness and efficiency.

Numerous libraries (or packages) exist for analyzing temporal and complex networks, spanning multiple programming languages such as R, Python, and Java [4–6]. However, Python stands out for its popularity, user-friendly syntax, and extensive library ecosystem, making it an ideal choice for both beginners and experienced programmers engaged in temporal network analysis [7]. Python's simplicity, combined with its powerful libraries, provides a comprehensive toolkit for handling temporal data, analyzing patterns, and visualizing data [8,9].

Although there are several reviews of Python libraries on different contexts [9–11], there has been limited focus on networks and their temporal dimension [12]. Existing reviews focus on data visualization [9] or specific applications, such as social media analysis [13]. To the best of our knowledge, there are no other reviews of Python's library for static or temporal networks.

This study addresses this gap by providing a comprehensive review of Python libraries for temporal networks, evaluating them based on criteria such as ease of use, functionality, popularity, and community support. By providing a detailed assessment of each tool's strengths and weaknesses, we aim to assist researchers and practitioners in selecting the most appropriate libraries for their specific analytical needs. This approach focuses on enhancing efficiency, ensuring high-quality results, and identifying gaps in existing libraries.

2. Background and related work

In recent years, the number of Python library reviews has grown significantly, reflecting the increasing popularity of Python across various domains. Several studies have provided comprehensive overviews of Python libraries across different applications [8,11,13–16]. For example, reviews focusing on data mining and big data analysis have been proposed [14], as well as those examining Python libraries and integrated development environments (IDEs) available for data science [11,15]. These studies provide valuable insights into the strengths and limitations of different tools, helping users make informed decisions based on their specific needs.

Moreover, there is growing interest in using Python for specialized applications, such as social networks, where surveys have identified the most effective libraries for extracting data from various platforms [13]. Another area that has received attention is data visualization, with studies comparing the capabilities and performance of popular libraries like Matplotlib and Seaborn [16], as well as broader assessments of Python's visualization tools [9]. These studies highlight the importance of selecting appropriate visualization tools to effectively analyze complex data and generate insights.

In addition to these areas, Python's role in time series analysis has been previously reviewed, providing an overview of the packages best suited for handling temporal data [8]. The impact of Python in artificial intelligence and machine learning has also been a major focus, with comprehensive reviews detailing the top Python-based deep learning packages [17] and evaluating the effectiveness of libraries specifically designed for machine learning tasks [10].

While these reviews cover a wide range of topics, there is a gap in the literature regarding Python libraries specifically designed for network or graph analysis, particularly those that incorporate temporal aspects [5,12]. Steer et al. [5] and Lucas et al. [12] discuss various Python libraries for temporal network analysis. They observe that while some tools address specific aspects, such as respecting paths and temporal slicing, they often suffer from limited functionality and scalability, or are still in early development [5].

These observations reveal a gap in the literature. Despite the individual strengths and weaknesses of various tools, and a clear interest in the literature in reviewing Python libraries for several contexts, there is a lack of comprehensive reviews that assess Python libraries for temporal networks to address their quality, such as popularity and specialized functionalities. This gap shows the necessity for a detailed review to guide researchers and practitioners in selecting the most suitable tools for their temporal network analyses.

3. Methodology

As part of the methodology, we first established inclusion and exclusion criteria, then divided the analysis into five perspectives: visualization, graph metrics, documentation, scoring, and format/functionality (see Fig. 1).

3.1. Inclusion and exclusion criteria for libraries

We established inclusion and exclusion criteria to identify the most relevant temporal Python libraries, ensuring our selection aligns with quality, maintenance, and functionality standards. A primary criterion for inclusion was the presence of functionalities devoted to temporal networks, which are crucial for our review.

Another criterion was how recently the libraries were updated. We prioritized libraries updated within the last 2 years (2023–2025) to ensure we are working with tools that incorporate the latest advancements and are compatible with current Python versions and dependencies. To verify this, we examined individual GitHub repositories and reviewed their release histories. We excluded libraries that had not been updated recently due to concerns about compatibility and maintenance. Older libraries may present compatibility issues with newer Python versions or other dependencies, potentially leading to

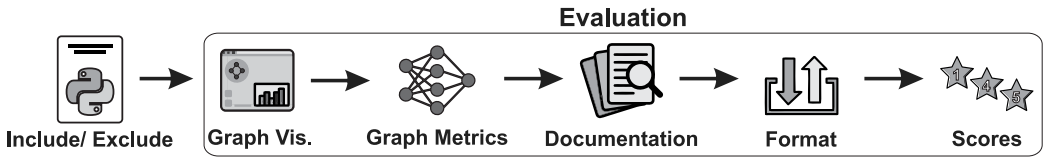

Fig. 1. Methodology workflow for evaluating libraries based on temporal network capabilities, documentation quality, and scores aspects.

deprecated syntax and conflicts. Such libraries might also lack active maintenance, resulting in missed bug fixes and updates that could undermine the reliability and effectiveness of our analysis. This two-year threshold is aligned with methodological practices in software and tool review studies, where recency of updates is used as an inclusion criterion to ensure that reviewed tools are actively maintained and compatible with current environments [18].

By applying these criteria, we ensure that our selection of libraries not only meets our immediate analytical needs but also guarantees long-term viability and support, focusing on those that are actively maintained and specifically designed for temporal network analysis.

3.2. Temporal network visualization

Temporal network visualization is useful for observing how connections evolve, revealing patterns and trends that could be hidden in the analysis. Tracking these changes helps identify dynamic behaviors and structural shifts. Inspired by Beck et al.'s taxonomy [19], which roughly categorizes visualization techniques into animation and timeline, we decided to analyze the libraries based on the visualization types they actually offer: Timeline (TL), Animated Node-link Diagrams (NL), or Small Multiples (SM). There are many visualization techniques in general, and we do not consider their pros and cons in our review.

Timeline (TL) usually depicts the nodes and their connections on a 2D plane, where the nodes/edges are drawn on a vertical axis and the time is mapped on the horizontal axis (see Fig. 2(A)). A different strategy employs the popular node-link diagram, which depicts nodes as circles and edges as straight lines. In the case of temporal networks, the time can be mapped using an animated node-link diagram (NL), where the network snapshot at a particular timestamp is visualized as an animation frame (see Fig. 2(B)). Finally, another popular visualization is small multiples (SM), which considers side-by-side layouts (images) to facilitate comparison. This last strategy is used in the context of temporal networks by associating each timestamp (network snapshot) with a different image in a sequential time ordering (see Fig. 2(C)).

3.3. Temporal network metrics

Graph metrics are essential for understanding both the structure and dynamics of networks. For instance, density measures the proportion of actual connections relative to all possible connections in the network, while centrality metrics identify important nodes within it. Examples of centrality metrics are: degree, which counts the number of direct connections a node has; betweenness, which measures how often a node lies on the shortest path between other nodes, indicating its role as a connector; and closeness, which reflects how quickly information can spread from a node to all others in the network. Moreover, the network's community structure is an important metric, as it indicates clusters or groups of nodes that are more connected to each other than to the rest of the network.

In temporal networks, where the network topology changes over time, traditional metrics like density, centrality, and community structure can be extended to account for these temporal variations [20]. For example, temporal density measures how connections evolve over time rather than just considering a static snapshot. Temporal centrality metrics, like temporal betweenness or closeness, assess the importance of nodes not only based on their overall connections but also on the timing of those connections—how influential a node is at specific moments in the network's evolution. Temporal community detection identifies groups of nodes that frequently relate within certain time frames, revealing how communities are born, die, merge, split, and so on [21]. This temporal analysis is crucial for understanding dynamic processes such as information flow, disease spread, or evolving social networks.

To organize the many existing temporal network metrics, we proposed five categories inspired by Thompson et al. [22]: Centrality Measures (CEM), Community-Dependent Measures (CDM), Global Measures (GLM), Edge Measures (EDM), and Community/ Group Measures (CGM). These categories helped us structure the evaluation of different Python libraries by grouping metrics based on their functions and roles in network analysis. Importantly, we focused solely on metrics (also referred to as measures in some contexts) rather than algorithms.

Centrality Measures (CEM) evaluate the importance or influence of individual nodes in a temporal network. Common examples include temporal degree centrality, which counts the number of direct connections a node has over time, and temporal betweenness centrality, which measures how often a node lies on the shortest paths between other nodes. These metrics help identify key nodes, which are crucial for understanding information flow and network stability.

Community-Dependent Measures (CDM) assess the structure of specific groups or communities within a temporal network. Metrics such as temporal participation coefficients and bursty coefficients reveal how nodes participate in or fluctuate within their respective communities over time. These metrics are particularly useful for understanding the temporal dynamics of groups in evolving networks.

Global Measures (GLM) provide an overview of the entire network's structural characteristics. Metrics such as temporal efficiency, which reflects how efficiently information is transferred across the network, and reachability latency, which indicates the time required for information to reach all nodes, offer insights into the network's global properties. These measures help assess the network's overall organization, connectivity, and responsiveness, contributing to a comprehensive understanding of its structural dynamics.

Edge Measures (EDM) focus on the relationships between nodes by analyzing temporal edges or connections. For example, shortest temporal paths measure the shortest paths between nodes over time, while intercontact times measure the time between consecutive relationships of two nodes. These metrics are essential for studying the dynamic nature of connections in temporal networks, including how quickly information or influence spreads.

Community/Group Measures (CGM) evaluate the properties of entire communities or groups within the network. Metrics like allegiance or persistence reveal how tightly connected groups are and how they evolve over time. These measures provide insights into how communities are created, dissolved, or change structure, offering valuable information on social or biological network dynamics.

By grouping the metrics into the five discussed categories, we can more effectively evaluate and compare their functionalities and applications in temporal network analysis. The libraries provide access to these metrics computation through implemented functions or methods. We evaluated these methods on a case-by-case basis to determine the appropriate category and ensure a comprehensive, standardized analysis. However, some cases needed to be filtered based on inclusion and exclusion criteria. Our inclusion criteria were based on whether the

Table 1
Documentation evaluation criteria for temporal network analysis libraries.

General documentation	Methods documentation
1. Is the documentation comprehensive and distributed into multiple pages?	5. Are the methods, parameters, and output explained?
2. Does the documentation provide background information?	6. Are there examples of usage?
3. Does the documentation explicitly provide contact information?	7. Are there references for temporal metrics?
4. Are all internal hyperlinks and pages working correctly?	8. Are there visual examples (figures) of drawing methods?

methods provided distinct and relevant temporal metrics that directly quantify graph properties at the node, edge, group, or global level. For example, metrics — such as temporal betweenness centrality, temporal efficiency, intercontact time, and community persistence — were included because they explicitly account for the timing or ordering of interactions and return quantitative values characterizing the evolving network.

In contrast, we excluded methods that do not directly quantify graph properties or that primarily represent algorithmic or structural procedures rather than metrics. Examples of excluded cases comprise methods that compute or enumerate temporal paths without aggregating the results into a scalar measure (e.g., returning a set of time-respecting paths or listing all shortest temporal paths without producing a summary statistic). We also excluded methods that return temporal listings or time series rather than a quantitative metric, such as functions that output nodes or edges per timestamp or connected components per time step. These methods provide useful structural information but do not directly quantify graph properties as metrics. The final selection was guided by the metrics' ability to directly measure graph attributes, ensuring that our analysis was both relevant and rigorous. This approach allowed us to categorize and evaluate the metrics effectively within the defined framework of temporal network analysis.

The counting of metrics per category was performed through a systematic, manual inspection of publicly available materials for each library. Specifically, we reviewed the official documentation, tutorials, and usage examples provided by the library maintainers to identify functions that compute temporal network metrics. Each identified metric was then assigned to one of the five categories based on its primary analytical role. This manual, case-by-case procedure ensured consistency across libraries and allowed us to reliably determine the number of metrics per category.

3.4. Documentation evaluation

We conducted an evaluation by reviewing the documentation for each library via hyperlinks on their official websites or in available repositories. Our assessment focused on eight questions divided into two categories (see Table 1). These questions aimed to ensure that each library's documentation is complete, user-friendly, and supportive, addressing the needs of researchers and practitioners in the field of temporal network analysis.

In line with studies that highlight the impact of completeness issues in software documentation [23,24], all questions aimed to evaluate the libraries' documentation on this matter from the user's macro perspective. Regarding the first set of questions (see "General Documentation" in Table 1), the first question assessed the overall comprehensiveness of the documentation, in terms of potentially crucial sections that might be missing, and whether the sections are distributed into multiple pages, which would reflect better information organization and overall usability [23]. The second question examined whether the documentation provides background information, which is essential for understanding complex concepts and methodologies in temporal network analysis. The third question looked at whether the documentation explicitly provides contact information, such as an email address, for users seeking assistance or clarification. The fourth question ensured that internal hyperlinks and pages within the documentation worked correctly, thus enabling better navigation and findability of useful information. Note that the presence of broken hyperlinks would also represent one of the most relevant issues in documentation up-to-dateness [23].

The second set of questions focused on the details of the methods related to temporal metrics or temporal visualization. The fifth question evaluated whether the documentation effectively explains the methods, parameters, and outputs, helping users understand the library's functionalities. The sixth question assessed whether the documentation includes practical examples of use, which is important for assisting users when using the library's features. The seventh question examined whether there are references for further information on metric-related methods. Finally, the eighth question reviewed whether the documentation includes figures illustrating the application of visualization methods, providing visual insights into their capabilities and potential uses.

3.5. Format evaluation

The evaluation of input/output formats and functionality was carried out by systematically assessing each library's ability to handle different data structures through its supported formats. The number of formats available was considered a crucial factor in determining a library's versatility, as broader format support enhances interoperability with various datasets and external tools. The presence of conversion functions was also examined to determine how easily users can transform data between different formats. Libraries with built-in conversion functions offer greater convenience and seamless data integration, making them more user-friendly and adaptable across diverse analytical workflows.

We also considered distinctive features that contribute to the overall usability and effectiveness of each library. These distinct aspects include access to built-in datasets, GPU acceleration, integration with scientific writing tools, and advanced analytics capabilities, such as temporal edge metrics and multilayer modeling. By highlighting these properties, the evaluation provides a comprehensive perspective on how each library supports different types of temporal network analysis, helping users choose the most suitable tool for their specific needs. The uniqueness criterion also considered scientific papers that explicitly apply these libraries, using these examples to contextualize their features within domain-specific applications reported in the literature.

3.6. Scores evaluation

We selected our scores evaluation criteria to provide a comprehensive assessment of each Python library's reliability, security, and community support. These criteria include several important factors. First, the software license defines the terms of use, modification, and distribution. Second, Snyk's score [25] is used to evaluate library security by identifying known vulnerabilities. Third, the number of downloads serves as an indicator of the library's popularity and community trust. Fourth, the last stable version published reflects how up-to-date and well-maintained the library is. Finally, GitHub activity offers insights into development activity and community engagement. Together, these factors provide a well-rounded perspective on the suitability of each library for our purposes.

The licenses we considered include MIT, GPL-3 (GNU General Public License version 3), AGPL-3 (GNU Affero General Public License version 3), and BSD-2 (Berkeley Software Distribution 2-Clause License). MIT and BSD-2 are permissive licenses that allow users considerable freedom to use and modify the software. In contrast, GPL-3 and AGPL-3 are more restrictive; GPL-3 requires modified works to be open source as well, while AGPL-3 imposes additional requirements for networked

applications. The choice of license can significantly influence user decisions based on their desired levels of freedom, obligations regarding modified works, and compatibility with other software licenses.

For our other scoring criteria, we considered the Snyk score, which provides insights into an open-source library's security, popularity, maintenance, and community support. However, we did not find detailed calculation methods for this score; the higher the score, the healthier the library. The number of downloads was collected using PEPy [26], a tool that tracks and visualizes download statistics by analyzing logs from PyPI's server. This logging occurs when a package (library) is installed using pip or any other package manager that fetches from PyPI. PEPy aggregates this data to provide download statistics for each package.

The last stable version indicates the release date of the most recent stable version for each library on pip, which is crucial for users as it signals whether recent updates or improvements have been made available. Lastly, the GitHub activity shows the date of the latest update to the library's repository, indicating ongoing development, bug fixes, and community engagement. A more recent activity date suggests active maintenance and continued development of the library.

4. Results

We discuss our results according to the steps outlined in our methodology (following the workflow shown in Fig. 1). The results are based on the libraries' information available in March 2025, when they were collected and validated.

4.1. Excluded libraries

In our selection process, we prioritized libraries specifically designed for temporal networks in Python, thereby excluding several well-known libraries that are either focused on static networks or have not received recent updates. One example is GraphTool [27], which is highly efficient and has been a cutting-edge tool in network science for over 15 years, but whose primary focus is on static networks. While GraphTool can be adapted to incorporate temporal information by treating time as an edge property, it does not inherently support temporal network features, making it less suitable for our needs. Similarly, igraph [4] performs well in the analysis of static networks but struggles with the dynamic nature of temporal graphs, limiting its applicability in our context of evolving network structures.

NetworKit [28] is another powerful toolkit designed for large-scale static network analysis, emphasizing parallel computation and scalability. However, its focus remains on static measures, and it lacks built-in support for temporal dynamics, which is crucial for our analysis. RecallGraph [29], though designed for temporal graph data, functions as a microservice for ArangoDB rather than as a Python library, and thus falls outside the scope of our evaluation despite its relevance to temporal data.

Additionally, libraries such as Stinger [30], which is optimized for massive streaming graphs with temporal and semantic information, and others, such as pyTempNets [31] and Tacoma [32], were not included due to their lack of updates over the past five years. Although these libraries once offered robust solutions for temporal networks, their inactivity raises concerns about compatibility and long-term support, both of which are critical to our analysis. Nevertheless, note that such libraries may still remain usable in controlled or legacy environments where dependencies are fixed and compatibility is ensured. In these contexts, tools like pyTempNets [31] and Tacoma [32] can still provide valuable functionality for temporal network analysis, particularly for reproducibility of earlier studies or historical comparisons. However, their limited maintenance makes them less suitable for general adoption in current workflows.

Finally, we did not consider Networkx in our final analysis. Networkx is the most renowned network library in Python, having been a foundational tool since its 2005 release [33]. With nearly a billion downloads and a Snyk score of 96 out of 100,[1] its widespread popularity is due to its long-standing support and active community. Despite its versatility and extensive features, Networkx is primarily designed for static networks and lacks built-in support for temporal network analysis. While it can also be adapted to handle temporal data by treating time as an edge property, it does not offer specialized metrics or visualizations for temporal networks. Therefore, our focus has shifted to libraries specifically designed for temporal networks, which are better suited to meet our analysis criteria.

4.2. Included libraries

After reviewing various libraries, we discarded those that did not meet our criteria and selected nine that were appropriate for our analysis. The selection was based on libraries with dedicated functionalities for temporal networks and recent updates. Libraries that had not been updated recently were excluded due to concerns about compatibility and maintenance (see more details in Section 3.1).

The limitations of Networkx for temporal networks have led to the development of extensions such as **Networkx-temporal** [34] (hereafter called Networkx-t) and **DyNetX** [35], which adapt Networkx for temporal network analysis. Networkx-t allows for converting temporal networks into formats compatible with other libraries and introduces a few temporal metrics. DyNetX is a Python package that extends Networkx for modeling and studying temporal networks. It provides tools for analyzing dynamic social, biological, and infrastructure networks in a flexible and collaborative development environment.

Raphtory [5] implements its own set of metrics and data structures for large-scale temporal networks. Raphtory stands out for its focus on scalability and the implementation of an infection model (SEIR). It also allows one to export to tools such as NetworkX and pyVis, enabling further exploration of the results. **PathpyG** [36] emphasizes interactive graph visualization embedded within Jupyter notebooks. It leverages torch and tensor-based representations for sparse graph models and supports causality-aware GNNs, making it ideal for analyzing time series data and calculating graph paths.

Teneto [22] provides a specialized toolset for analyzing temporal networks, particularly important in neuroimaging contexts where the temporal evolution of connectivity patterns is essential. **TGX** [37] focuses on automating the analysis of temporal networks, offering a streamlined pipeline for data loading, processing, and network analysis. TGX distinguishes itself with features like dataset integration from Temporal Graph Benchmark and functionalities for handling large datasets, such as node subsampling and temporal graph discretization.

Reticula [38] introduces a unique capability by supporting both static and temporal hypergraphs (networks in which an edge can involve more than two nodes), positioning itself as a general-purpose tool for complex network analysis. Meanwhile, **Phasik** [12] takes a different approach, focusing on timestamp clustering and offering tools to identify and visualize temporal phases across multiple scales, making it particularly useful for time series analysis. Finally, **TGLib** [39] is a dedicated library for computing temporal distances and centrality measures in temporal networks. It is an open-source C++ template with a Python front-end that provides powerful computational capabilities, thanks to its potentially high performance.

4.3. Visualization and metrics aspect

Table 2 presents the nine libraries evaluated according to the available temporal visualizations and metrics. From the visualization perspective, we consider Timelines (TL), Animated Node-Link Diagrams (NL), and Small Multiples (SM), cf. Section 3.2. In the case of small

[1] https://snyk.io/advisor/python (Accessed: 03.02.2025)

Table 2
Temporal Visualization and metrics aspect. The visualization options are Timeline (TL), Animated Node-link Diagram (NL), and Small Multiples (SM). Metrics are Centrality Measures (CEM), Community-Dependent Measures (CDM), Global Measures (GLM), Edge Measures (EDM), and Community/Group Measures (CGM).

Libraries	Temporal Visual.			Temporal Metrics				
	TL	NL	SM	CEM	CDM	GLM	EDM	CGM
Teneto [22]	✓		✓*	5	5	6	3	6
Networkx-t [34]			✓	1	0	1	1	0
PathpyG [36]		✓		2	0	0	0	0
TGX [37]				3	0	4	2	0
Raphtory [5]			✓*	0	0	0	1	3
DyNetX [35]			✓*	0	0	0	1	0
Reticula [38]				–	–	–	–	–
Phasik [12]		✓		–	–	–	–	–
TGLib [39]				7	0	3	0	0

multiples, we consider libraries that explicitly place node-link diagrams side-by-side, as well as those that provide a single node-link diagram (the latter marked as *), since a set of these can be intuitively converted to small multiples. Only three libraries do not present any form of visualization (TGLib, Reticula, and TGX).

Beyond the availability of visualization types, the libraries differ substantially in terms of visual quality and customizability. Teneto, for instance, provides built-in support for timelines and node-link visualizations but offers limited control over visual encoding. Customization is mainly limited to selecting predefined colormaps, with no explicit support for mapping visual attributes (e.g., color, size, or opacity) to node- or edge-level measures. This limits its flexibility for tailored visual analysis, although it remains effective for rapid exploratory inspection of temporal structures. In contrast, Networkx-t allows users to adjust layout-related parameters, such as force-directed algorithms (e.g., Kamada–Kawai), enabling some control over node organization, but it provides fewer options for fine-grained visual styling beyond layout selection.

Other libraries emphasize richer visual customization and interactivity. PathpyG offers interactive node-link visualizations with extensive styling options, including explicit control over node and edge size, color, opacity, and attribute-based mappings. Styling can be applied globally or per element, supporting both uniform encodings and index-based or attribute-driven visual differentiation. Phasik similarly provides a range of parameters for animated temporal visualizations, allowing users to adjust colors for temporal and constant edges, edge widths, animation speed, labeling, and node positioning.

Raphtory supports visualization through PyVis integration, offering customizable parameters such as node shapes, edge colors, labels, weights, and directionality, particularly suited for interactive and notebook-based exploration. In contrast, DyNetX offers basic visualization functionality but exposes little to no documented support for visual customization. Overall, while the TL, NL, and SM classification captures visualization availability, these differences in expressiveness and configurability are crucial in practice, as they directly affect the suitability of each library for exploratory analysis, presentation-quality figures, and interactive analytical workflows.

Solely Teneto presented more than one type of visualization, and it was also the only library to present a timeline representation (as illustrated in Fig. 2(A)). In that sense, for visualization purposes, Teneto offers the best cost-benefit, as it provides two very popular visualization types. Nevertheless, if an animated node-link diagram is required, PathpyG or Phasik (as illustrated in Fig. 2(B)) should be the chosen ones. Lastly, if the requirement is to work with small multiples, Networkx-t should be the first choice since it provides a native implementation of this visualization (as illustrated in Fig. 2(C)). If the set of possible visual styling tasks for nodes/edges (such as changing colors, shapes, strokes, and sizes) reaches a limit from the user's perspective, Teneto, Raphtory, and DyNetX should be considered.

To provide a more structured comparison of these aspects, Table 3 summarizes the main differences in customization, interactivity, and output quality across the libraries. It is also important to note that some libraries (TGX, Reticula, and TGLib) do not provide native visualization support. The customization classification (low, medium, high) is based on the degree of control over visual attributes such as node and edge color, size, opacity, layout, and the ability to map these properties to data-driven measures. Libraries classified as low on customization (e.g., Teneto and DyNetX) offer only limited styling options, typically restricted to predefined parameters or basic visual outputs. Medium customization (e.g., Networkx-t, Phasik, and Raphtory) reflects partial control, such as layout configuration, animation parameters, or integration with visualization frameworks, but without extensive support for attribute-based styling. High customization (PathpyG) corresponds to libraries that allow fine-grained, element-level control and flexible mappings between data attributes and visual properties.

Interactivity is evaluated based on whether the library supports dynamic exploration, such as animations or user interaction within visualizations, while output quality reflects the suitability of the generated visualizations for exploratory analysis versus presentation-ready figures. Overall, PathpyG stands out as the most versatile library, combining high customization with interactive capabilities and support for both exploratory and presentation contexts. In contrast, Teneto and DyNetX are less expressive visually, making them better suited for quick exploratory analysis rather than detailed or publication-quality visualizations.

Before evaluating the temporal network metrics, we had to review the methods from each library individually to guarantee a thorough and consistent analysis. Reticula, for instance, primarily offers methods that are not strictly metrics but rather enable structural analyses or network manipulations. We excluded these methods because they do not directly quantify properties in accordance with our defined criteria. In contrast, Raphtory includes methods similar to Reticula but provides a measure of node importance within a temporal context, aligning with our criteria. Thus, Raphtory was considered in our analysis.

Phasik methods were also not included as they focus on temporal phases over multiple scales rather than providing explicit metrics. Such a focus on temporal phases does not align with the metric categories we consider. Likewise, some TGLib methods were not considered because they focused on identifying structural components rather than measuring graph properties. We aimed for metrics that quantify graph attributes directly, rather than categorizing or decomposing networks into structural components. At last, we considered only some methods from DyNetX, Networkx-t, and PathpyG. We limited our consideration to their native temporal metrics and excluded methods that rely heavily on Networkx functions unless they provided novel or additional insights beyond those available in Networkx itself.

Table 2 includes the number of implemented temporal metrics according to the proposed five categories (CEM, CDM, GLM, EDM, and CGM – see Section 3.3 for details of each category). Teneto leads in most metrics across most categories, including critical metrics such as temporal degree centrality and volatility, which are essential for analyzing node importance and community dynamics over time. TGLib

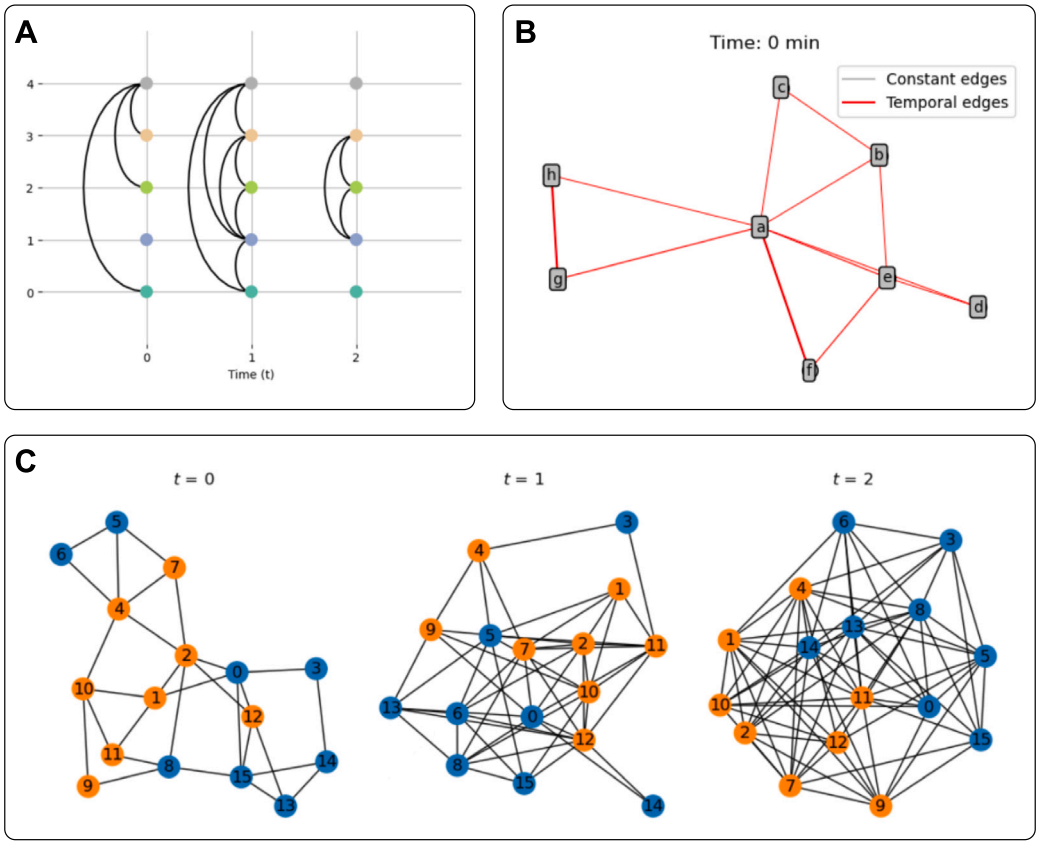

Fig. 2. Examples of temporal visualizations: **(A)** Timeline (TL); **(B)** Animated Node-Link Diagram (NL); and **(C)** Small Multiples (SM).

Table 3
Comparison of visualization customization, interactivity, and output quality.

Libraries	Customization	Interactivity	Output quality
Teneto [22]	Low	Low	Medium
Networkx-t [34]	Medium	Low	Medium
PathpyG [36]	High	High	High
TGX [37]	–	–	–
Raphtory [5]	Medium	High	Medium
DyNetX [35]	Low	Low	Low
Reticula [38]	–	–	–
Phasik [12]	Medium	Medium	Medium
TGLib [39]	–	–	–

excels in the CEM category, with metrics such as temporal closeness and Katz centrality, offering deep insights into node accessibility and influence.

PathpyG and Networkx-t provide fewer metrics, but their methods, such as time-respecting paths and temporal order, add valuable perspectives by integrating static Networkx functions into temporal analysis. Reticula and Phasik lack metrics in Table 2 because they focus on different, often specialized tasks (e.g., Phasik's focus on temporal phases across multiple scales). The diverse range of metrics available across these libraries highlights the flexibility and depth of temporal network analysis, allowing researchers to choose tools that best meet their specific needs and ensuring a comprehensive understanding of temporal dynamics.

Beyond the raw counts shown in Table 2, some libraries stand out due to the presence of distinctive and analytically valuable temporal metrics. Teneto offers a particularly diverse set of measures that go beyond classical temporal centrality, including volatility, flexibility, integration, and allegiance, which enable the study of temporal stability, role switching, and community affiliation dynamics over time. These metrics are especially valuable for analyzing evolving communities and fluctuating interaction patterns, offering perspectives largely absent from other libraries. TGLib, on the other hand, distinguishes itself by a strong focus on temporal centrality measures, such as temporal Katz, temporal PageRank, and temporal walk centrality, which extend well-established influence-based concepts to temporal settings and enable fine-grained assessments of node importance under time-respecting constraints. While Networkx-t and PathpyG offer fewer metrics overall, their emphasis on concepts like temporal order and time-respecting paths provides essential building blocks for temporal reachability analysis.

In summary, when evaluating the libraries based on visualization capabilities, Teneto emerges as the most versatile, particularly for those requiring timeline representations and small multiples. PathpyG and Phasik are excellent choices for tasks involving animated node-link diagrams, while Networkx-t is the best option for native small multiples. Regarding metrics, TGLib surpasses Teneto in Centrality Measures, making it the top choice for detailed centrality analysis. However, when considering both visualization and metrics together, Teneto remains the leading option, offering a balanced combination of popular visualization strategies and a comprehensive set of temporal

Table 4
Documentation analysis divided into general and methods-related: (1) Comprehensive, multipage documentation; (2) Background information; (3) Contact information; (4) Working hyperlinks; (5) Explanation of methods/parameters; Contains (6) usage examples, (7) references, and (8) figures. The symbol "✓" stands for "fully covered by the criterion"; "/" stands for "partially covered".

Libraries	General Document.				Methods Document.			
	1	2	3	4	5	6	7	8
Teneto [22]	✓	✓		✓	✓	/	/	✓
Networkx-t [34]				✓	✓	✓		✓
PathpyG [36]	✓		✓	✓	✓	✓	✓	✓
TGX [37]	✓		✓	✓	✓	✓	/	✓
Raphtory [5]	✓		✓	✓	✓	/	/	✓
DyNetX [35]	✓			✓	/	/		
Reticula [38]	✓			✓	/	/	/	
Phasik [12]	✓	✓	✓	✓	✓	/	✓	/
TGLib [39]			✓	✓			✓	

metrics. At last, TGLib and TGX provide a variety of temporal metrics but no visualizations.

4.4. Documentation evaluation

Table 4 presents the summary of the analysis of the libraries' documentation according to the questions described in Section 3.4. In the table, a green checkmark (✓) indicates that the library fully satisfies the corresponding criterion. The symbol "/" denotes partial coverage, meaning that some parts of the documentation meet the criterion while others do not; for instance, when not all methods and parameters are explained, or when only a subset of visualization features is accompanied by illustrative figures.

Every evaluated library provides working hyperlinks (4), making this a consistently covered criterion across them. Almost all libraries also offer comprehensive, multipage documentation (1), except for Networkx-t and TGLib, which provide more limited or summarized information in their documentation. Background information (2) and contact information (3) are less universally covered. These gaps in documentation across libraries highlight variations in overall user support and raise points for improvement.

Teneto and Phasik stand out for the quality of their documentation. Both libraries cover almost every aspect, except for contact information (3) in Teneto, and some points that are partially covered in both cases. In the case of Teneto, for example, its documentation provides code examples (6) on how to compute some of the metrics (e.g., topological overlap and inter-contact times); however, not all metrics are covered (e.g., temporal degree and reachability latency).

PathpyG and TGX also perform well, covering key areas. PathpyG stands out for its completeness, meeting all aspects except background information (2). TGX, though strong overall, lacks background information (2) and only partially provides references for further information about the metrics (7). As an example of this partial coverage, even though the same Ref. [40] brings information about Novelty index, Surprise index, Reoccurrence index, Temporal Edge Appearance, and Temporal Edge Traffic (all mentioned in TGX documentation), this reference is only mentioned in the context of the latter two. For the first three metrics, no reference is provided.

Networkx-t, Reticula, and DyNetX share some limitations. None of these libraries provides background (2) or contact information (3), potentially reducing user understanding and support. Networkx-t and DyNetX also do not provide references for further information about the metrics (7). Overall, Reticula and TGLib are the least robust in documentation. Reticula, while partially covering three of the four criteria for methods documentation (5, 6, 7) and lacking the fourth one (8), also lacks background (2) and contact information (3). All these limitations greatly impair the quality of the documentation and user support. TGLib falls short, offering only contact information (3), hyperlinks (4), and references for further information (7), with nothing else to guide users through its features.

Based on our documentation evaluation, Teneto, Phasik, PathpyG, and TGX emerge as the top-performing libraries, providing complete documentation across almost every aspect. Phasik lacks some coverage of figures, and Teneto is missing contact information, but both remain strong options. Libraries like Networkx-t and Raphtory fall in the middle range, offering reasonable coverage but with gaps in background and contact information. Reticula, DyNetX, and TGLib stay behind, offering the least documentation, particularly lacking in background, figures, and examples.

4.5. Input/output format evaluation

Table 5 presents the nine libraries evaluated based on their input and output formats, conversion options, and distinctive properties. Among them, Networkx-t, PathpyG, and TGX support the most input and output formats, making them the most versatile. In contrast, DyNetX, TGLib, Reticula, and Phasik offer only two input or output formats, limiting their flexibility. The table also provides details on conversion functions that facilitate data transformation between formats and highlights the aspects of each library.

A key comparison factor is the variety of supported input and output formats. Most libraries commonly support edge lists and adjacency matrices as input and output formats. CSV and JSON formats are also widely supported, ensuring integration with general-purpose data processing tools. Pandas DataFrames are used as input and output formats by PathpyG and Raphtory. Also, JSON is supported by three of the nine libraries: Networkx-t, PathpyG, and Raphtory. In turn, CSV is supported by four libraries: PathpyG, TGX, Raphtory, and Phasik. Exclusive formats set certain libraries apart. For instance, PathpyG supports LaTeX as an export format, enabling direct incorporation into scientific documents.

Some libraries, such as Reticula, support event-list formats, which are particularly useful for temporal network analysis. Additionally, visualization tool compatibility is an important distinction: Networkx-t and Reticula can output data in formats suitable for Gephi, a widely used graph visualization software [41]. DyNetX is built upon NetworkX. Thus, the development of this library is influenced by NetworkX. Furthermore, DyNetX supports reading and writing networks in two formats: snapshot and interaction graph.

Conversion functions are crucial for compatibility between formats. PathpyG and Raphtory provide multiple conversion options, enhancing data adaptability. Raphtory, for example, supports conversion between various structured data types, such as Pandas DataFrames and CSV files. On the other hand, TGX, Reticula, and TGLib do not provide conversion functions, limiting their interoperability with external tools. This limitation can reduce the integration with other workflows, making these libraries less flexible.

Beyond format support and conversion capabilities, each library has distinct features that define its use cases. Teneto efficiently handles dense, sparse, and weighted networks, making it particularly useful for

Table 5
Summary of key features of different libraries for temporal network analysis, including input/output dimensions, conversion capabilities, and unique functionalities.

Libraries	Input	Output	Conv.	Uniqueness
Teneto [22]	4	2	✓	Handles dense, sparse, and weighted networks.
Networkx-t [34]	10	7	✓	Supports large, complex temporal networks.
PathpyG [36]	6	6	✓	Optimal for Scientific Writing, high performance.
TGX [37]	19	3	✗	Supports all datasets from TGB.
Raphtory [5]	4	5	✓	Fast, memory-efficient, supports time-traveling.
DyNetX [35]	2	2	✓	Development environment for multidisciplinary projects.
Reticula [38]	2	2	✗	Fast and efficient analysis of temporal networks and temporal hypergraphs.
Phasik [12]	2	0	✓	Infer temporal phases in temporal networks.
TGLib [39]	2	2	✗	Performance and usability, Global Temporal Graph Statistics.

network-based statistical modeling. Also, users can insert new information at any time, and it will be automatically inserted in chronological order. Networkx-t excels at analyzing large-scale networks with built-in support for graph algorithms, centrality measures, and community detection. When analyzing high-performance libraries, PathpyG, TGLib, and Reticula stand out from the others. Beyond that, PathpyG is an optimal option for scientific writing, and TGLib offers several temporal graph statistics. The main advantage of Reticula lies in its ability to perform fast and efficient analysis of temporal networks and temporal hypergraphs, which makes it particularly useful for handling dynamic, time-evolving relationships within networks [38].

TGX stands out for offering compatibility with all datasets from the Temporal Graph Benchmark (TGB).[2] Also, users have access to statistical analysis and additional metrics, such as Temporal Edge Appearance (TEA) and Temporal Edge Traffic (TET) [37]. Raphtory is unique in its memory-efficient computations and advanced analytics features, including time-traveling and multilayer modeling [5]. DyNetX serves as a versatile development environment for multidisciplinary projects, and Phasik focuses on identifying temporal phases in temporal networks. This makes it particularly useful for analyzing how network behavior evolves over time [12].

Understanding the differences among libraries is essential for selecting the most suitable library based on specific research needs. Networkx-t, PathpyG, and TGX are the most versatile due to their broad format support and extensive conversion options. This versatility makes them ideal for users who need to adapt to diverse datasets. Teneto and Reticula offer specialized analytical capabilities, while others, such as Phasik and TGLib, have narrower applications due to limited format and conversion support. Notably, PathpyG stands out for its focus on scientific writing, offering high-performance capabilities that make it an excellent choice for researchers preparing publication-quality results and visualizations.

Beyond technical features, the uniqueness of each library was also considered through its domain-specific adoption, which provides insight into how its design choices translate into practical research use. Teneto, for example, has been extensively applied in neuroscience for time-resolved functional brain connectivity analysis, including studies on dynamic brain network reconfiguration during evoked pain in rheumatoid arthritis patients [42], the transition from static to temporal network theory in functional connectivity [43], and temporal network instability in disorders of consciousness following severe brain injury [44]. These applications reflect Teneto's distinctive strength in supporting fine-grained temporal analysis of neuroimaging data and help contextualize its differences beyond purely methodological characteristics.

[2] https://tgb.complexdatalab.com/docs/dataset_overview (Accessed: 03.02.2025)

Other libraries, such as PathpyG, have been applied in domains where higher-order temporal dependencies are central, such as human mobility and complex temporal processes, supporting higher-order network modeling of memory-driven mobility dynamics [45] as well as exploratory analysis and visualization of time series data on networks [46]. Raphtory has demonstrated applicability in large-scale socio-economic and streaming settings, where it has been used to measure social mobility in temporal networks [47]. In contrast, the remaining libraries (Reticula, Networkx-t, TGX, Phasik, and TGLib) currently appear less frequently in domain-specific publications, with their adoption primarily documented through official tutorials, example notebooks, and software repositories. Overall, these observations suggest that while several libraries remain largely methodological, others are already embedded in applied research domains such as neuroscience, mobility analysis, and socio-economic systems, and broader domain adoption is likely to increase as these tools mature.

Based on our evaluation of format support, TGX, Networkx-t, and PathpyG emerge as the top-performing libraries, offering extensive format compatibility and versatile conversion options across a wide range of temporal network types. These libraries provide robust flexibility, making them ideal choices for a variety of research needs. Beyond that, Raphtory, Teneto, and DyNetX offer medium-level performance solutions with limited input/output and very limited conversion options, although Teneto and Raphtory present really interesting domain applications. TGLib, Reticula, and Phasik fall below the middle range, offering minimal input/output formats.

4.6. Scores evaluation

Table 6 compares the analyzed libraries on practical adoption and sustainability indicators, including license type, security score (Snyk), download volume, recency of stable releases on `pip`, and repository activity on GitHub. As shown in Table 6, if we order the libraries from those with more permissive licenses to more restrictive ones, Networkx-t, TGX, Reticula, DyNetX, and TGLib use licenses such as MIT and BSD-2 that allow users to freely use, modify, and distribute the software with minimal restrictions. These permissive licenses facilitate both open-source and commercial use. In contrast, libraries licensed under GPL-3, such as Teneto, Raphtory, and Phasik, impose stricter requirements. These licenses ensure that derivative works also remain open source, which can be beneficial for maintaining software freedom but may limit commercial use and compatibility with proprietary software. Lastly, PathpyG, licensed under AGPL-3, is the most restrictive regarding network use. It mandates that users who interact with the software over a network must also provide access to the modified source code, adding an extra layer of openness that can be challenging for commercial applications.

Table 6
Scores analysis related to factors such as reliability, availability, and community support. Higher values are highlighted in bold. Information was collected in March 2025. The PathpyG numbers were considered the PathPy numbers, since PathpyG has not been released yet (March 2025).

Libraries	License	Snyk	Downloads	Last stable version on PyPI	Github
Teneto [22]	GPL-3	45/100	78,970	02/2021	11/2023
Networkx-t [34]	MIT	54/100	24,432	12/2024	01/2025
PathpyG [36]	AGPL-3	45/100*	34,678*	–	01/2025
TGX [37]	MIT	44/100	1852	02/2024	12/2024
Raphtory [5]	GPL-3	**78/100**	**367,778**	01/2025	03/2025
DyNetX [35]	BSD-2	46/100	**682,343**	06/2023	06/2024
Reticula [38]	MIT	57/100	85,245	11/2024	11/2024
Phasik [12]	GPL-3	35/100	16,628	11/2023	12/2023
TGLib [39]	GPL-3	**60/100**	7104	01/2025	01/2025

The score provided by *Snyk* offers insights into each library's overall health and security. Libraries with higher scores generally have better security practices and fewer vulnerabilities. In Table 6, we observe a range of scores from 35 to 78. Libraries with higher scores, such as Raphtory, TGLib, Reticula, and Networkx-t, indicate a relatively better security posture. In comparison, those with lower scores, such as TGX, Phasik, Teneto, and PathpyG, may require further attention to address potential security vulnerabilities. Users should consider these scores when evaluating a library's suitability and reliability for their needs, particularly for longer-term projects.

The release date of the *last stable version* on the package manager pip provides insight into the currency and ongoing development efforts of each library. Libraries that receive frequent updates and releases are generally considered to be actively maintained and improved. Conversely, libraries with older stable versions may indicate slower development or potentially abandoned projects. Table 6 shows some libraries having recent stable versions (e.g., Networkx-t, Reticula, TGLib, and Raphtory in late 2024/2025), some with a bit older releases (e.g., DyNetX and Phasik in 2023), one in 2021 (e.g., Teneto), and PathpyG with no actual releases since it has not been released yet (as of March 2025).

The date of the latest activity in the GitHub repository associated with each library reflects ongoing development, bug fixes, and community engagement. A more recent date suggests that the library is actively maintained and developed by its contributors. In Table 6, we observe that most have very recent GitHub activity (i.e., in 2024/2025), with only Teneto and Phasik having activities in late 2023.

In this scores assessment, Raphtory emerges as the top choice. It is GPL-3 licensed, ensuring open-source continuity. With the highest score in the table, Raphtory showcases robust security practices. Active maintenance, recent stable versions, and high downloads affirm its popularity and reliability. DyNetX ranks next, boasting BSD-2 licensing and a moderate score. Although older, it remains actively developed, offering versatility for network analysis. One library with high potential for success is TGLib, which was recently released and has strong initial scores and open-source potential. Networkx-t and Reticula follow closely behind Raphtory and DyNetX, all of which are under MIT licenses. Their moderate scores suggest reasonable security levels, with ongoing maintenance and recent updates in 2024/2025. Phasik has a more restrictive GPL-3 license, weak scores, and fewer recent updates. However, Teneto's stricter GPL-3 license and lack of recent stable versions raise concerns about its suitability, despite its relatively high number of downloads, which indicate significant popularity. PathpyG, under the AGPL-3 license, faces similar challenges, with a lower score and no recent update information. Users must evaluate its suitability carefully.

4.7. Summary evaluation

This summary provides a comprehensive comparison of the libraries, highlighting their strengths and weaknesses across different criteria. Table 7 shows our overall assessment for each library. The evaluation results reveal that almost all libraries were positive for at least one criterion (except Reticula), indicating that each has distinct advantages or strengths. TGX, Teneto, and PathpyG stand out as the best performers, as they were recommended in three of five criteria. Networkx-t is also a good choice in general, with balanced performance (two positive aspects and no negative ones) across all criteria, making it a safe, general-purpose option.

The symbols used in Table 7 represent a relative, qualitative assessment grounded in the comparative discussions presented in the preceding sections. A green checkmark (✓) indicates that a library clearly stands out as one of the strongest or most representative examples for a given criterion, as explicitly highlighted in the concluding paragraph of that criterion's subsection (e.g., Teneto for visualization versatility or TGLib for temporal centrality metrics). A dash (–) denotes regular performance, meaning that the library adequately supports the criterion and provides usable functionality, but does not distinguish itself as a leading or particularly strong example when compared to other libraries. Finally, a cross (✗) indicates poor performance, corresponding to cases in which the criterion is only minimally supported, largely absent, or not aligned with the evaluation's scope. This relative classification emphasizes comparative strengths rather than absolute thresholds, allowing the table to summarize how well each library meets each criterion relative to the others.

The middle-level group contains the libraries Raphtory and DyNetX. Raphtory had no negative sides in any criteria, but on the other hand, had only one recommended point, indicating a middle group. DyNetX was recommended by the scoring aspect and had two neutral aspects (format and visualization), even though it has two negative aspects (metrics and documentation), justifying its ranking as a middle-level quality library. These middle-level libraries indicate clear cases of highly specialized libraries with very specific strong points, but they can also entail significant trade-offs.

Finally, the worst-performing libraries were TGLib, Phasik, and Reticula, each with three or four negative aspects. TGLib and Phasik performed very well in two areas, but each had three negative aspects, indicating a low overall performance. At last, Reticula was the only library that did not perform well in any criteria evaluated.

This analysis shows that no single library performs best across all criteria. The varied strengths and weaknesses underscore the challenge of making a generalized recommendation. Each library offers distinct features, making it difficult to identify a universally superior option. Therefore, the choice of a library for temporal network analysis should be tailored to the user's specific needs and priorities, with the understanding that trade-offs are inevitable.

5. Limitations

In reviewing Python libraries for temporal network analysis, several limitations must be acknowledged. Our review focused predominantly on libraries that offer temporal network visualizations and

Table 7
Summary of evaluation results for each library across different criteria. The symbol "✓" indicates the library performed best in the respective criterion, "–" means regular performance, and "✗" indicates poor performance.

Libraries	Visualization	Metrics	Documentation	Format	Scores
Teneto [22]	✓	✓	✓	–	✗
Networkx-t [34]	✓	–	–	✓	–
PathpyG [36]	✓	–	✓	✓	✗
TGX [37]	✗	✓	✓	✓	–
Raphtory [5]	–	–	–	–	✓
DyNetX [35]	–	✗	✗	–	✓
Reticula [38]	✗	✗	✗	✗	–
Phasik [12]	✓	✗	✓	✗	✗
TGLib [39]	✗	✓	✗	✗	✓

metrics. Consequently, we did not consider libraries that provide algorithms for other analytical purposes, such as clustering or path-finding algorithms. This narrow focus may exclude valuable tools that, although outside our scope, could offer complementary functionalities for comprehensive network analysis.

Also, we did not evaluate the computational efficiency, scalability, or time complexity of the methods provided by these libraries. Although computational performance is critical for handling large-scale temporal networks, a detailed benchmarking assessment is outside the scope of this review. A fair performance comparison would require standardized datasets, controlled experimental setups, and consistent implementation of identical metrics across libraries, which would significantly expand the study's objectives and length. Therefore, our focus remains on comparing the libraries' functional coverage, temporal metrics, and visualization capabilities. We recommend that future work conduct dedicated performance evaluations to complement the findings of this review.

Aiming to help researchers and practitioners, Aghajani et al. [23,24] categorize problems in software documentation content into Correctness, Completeness, and Up-to-dateness issues. Our analysis of the libraries' documentation (see Section 3.4) primarily focused on completeness from the user's macro perspective. While including issues from the other categories would certainly strengthen this work, such analyses would be considerably more demanding, as many would require manual and detailed inspection of source code or comparisons across different versions of the same library. We intend to perform these new investigations in a future extension of this work.

Another limitation of this study is the limited exploration of visual styling capabilities. While we reviewed the visualization features offered by the libraries, we did not explore deeply how extensively users can modify the visual attributes of nodes and edges, such as colors, shapes, and sizes. Additionally, an important aspect we did not address is the role of interactive features in improving the quality and usability of visual layouts [48]. Interactivity and customization are essential for refining visualizations, as they can greatly enhance the clarity, comprehension, and overall effectiveness of temporal network analysis and presentation.

Moreover, our focus on Python libraries exclusively may limit the scope of our review. By not considering libraries from other programming languages, we may overlook powerful tools and methods that could offer superior functionality or performance for temporal network analysis. Expanding the review to include libraries from other languages could provide a more comprehensive overview of available tools and technologies.

6. Conclusion and future work

This study presents a comprehensive review of Python libraries for temporal network analysis, identifying tools that provide robust functionality for managing and interpreting dynamic network data. Our review focused on libraries that have been actively maintained and recently updated, ensuring inclusion of current, relevant tools.

Our analysis of Python libraries for temporal network analysis identified Teneto, Networkx-t, Phasik, and PathpyG as leading choices for visualization, with Teneto offering versatile temporal network visualizations. For metrics, Teneto stands out with its extensive options, while TGX and TGLib are also interesting choices. Teneto, PathpyG, Phasik, and TGX were noted for their comprehensive documentation, which prioritizes support, clarity, and ease of use. At last, the only two libraries that stood out in our scores evaluation were Raphtory and DyNetX. Overall, Teneto presented the best results, followed by Raphtory and Networkx-t. Based on the libraries' characteristics, we have also highlighted gaps and limitations that can motivate further improvements or even the creation of new libraries.

In future work, we intend to extend this study to a broader scope that includes various methods beyond metrics, a detailed evaluation of computational efficiency, an in-depth analysis of visual styling options, and a consideration of libraries across different programming languages. We believe addressing these areas in future reviews will provide a more complete understanding of the capabilities and limitations of temporal network analysis tools.

In addition, future work can include a qualitative assessment of expected scalability based on underlying implementation choices (e.g., compiled backends such as C++ or GPU-enabled frameworks such as PyTorch), providing further practical insight into how these libraries may perform in large-scale or high-frequency temporal network scenarios. Moreover, future work can also consider practical usability factors, such as ease of installation, onboarding experience, and community responsiveness (e.g., issue resolution time), which are important for adoption and long-term use of these libraries.

CRediT authorship contribution statement

Claudio D.G. Linhares: Writing – review & editing, Writing – original draft, Visualization, Validation, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Jean R. Ponciano:** Formal analysis, Data curation. **Martim R. Oliveira:** Writing – original draft, Validation, Investigation. **Amilcar Soares:** Writing – original draft, Validation, Conceptualization. **Agma J.M. Traina:** Writing – review & editing, Validation, Supervision. **Andreas Kerren:** Writing – review & editing, Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially supported through the ELLIIT environment for strategic research in Sweden. Claudio Linhares thanks the Swedish Research Council (Vetenskapsrådet), with registration number 2025-06537, for partially supporting this work. Jean R. Ponciano thanks the University of São Paulo (PRPI Ordinance No. 1032, Process 22.1.09345.01.2) for the financial support, as well as the Brazilian funding agencies FAPESP, Brazil Process 2024/13328-9, CNPq, Brazil, and CAPES, Brazil.

Data availability

No data was used for the research described in the article.

References

[1] L. da Fontoura Costa, O.N.O. Jr., G. Travieso, F.A. Rodrigues, P.R.V. Boas, L. Antiqueira, M.P. Viana, L.E.C. Rocha, Analyzing and modeling real-world phenomena with complex networks: a survey of applications, Adv. Phys. 60 (3) (2011) 329–412, http://dx.doi.org/10.1080/00018732.2011.572452.

[2] P. Holme, J. Saramäki, Temporal Network Theory, Springer Cham, 2019, p. 375, http://dx.doi.org/10.1007/978-3-030-23495-9.

[3] P. Holme, J. Saramäki, Temporal networks, Phys. Rep. 519 (3) (2012) 97–125, http://dx.doi.org/10.1016/j.physrep.2012.03.001.

[4] G. Csardi, T. Nepusz, The igraph software package for complex network research, InterJournal Complex Systems (2005) 1695.

[5] B. Steer, N. Arnold, C.T. Ba, R. Lambiotte, H. Yousaf, L. Jeub, F. Murariu, S. Kapoor, P. Rico, R. Chan, L. Chan, J. Alford, R.G. Clegg, F. Cuadrado, M.R. Barnes, P. Zhong, J.N.P. Biyong, A. Alnaimi, Raphtory: The temporal graph engine for rust and python, 2024, arXiv:2306.16309.

[6] R. Posenato, CSTNU tool: A java library for checking temporal networks, SoftwareX 17 (2022) 100905, http://dx.doi.org/10.1016/j.softx.2021.100905, URL https://www.sciencedirect.com/science/article/pii/S2352711021001564.

[7] K. Srinath, Python–the fastest growing programming language, Int. Res. J. Eng. Technol. 4 (12) (2017) 354–357.

[8] J. Siebert, J. Groß, C. Schroth, A systematic review of packages for time series analysis, Eng. Proc. 5 (1) (2021) http://dx.doi.org/10.3390/engproc2021005022, URL https://www.mdpi.com/2673-4591/5/1/22.

[9] A. Lavanya, L. Gaurav, S. Sindhuja, H. Seam, M. Joydeep, V. Uppalapati, W. Ali, V.S. SD, Assessing the performance of python data visualization libraries: a review, Int J Comput. Eng Res Trends 10 (1) (2023) 29–39.

[10] H. Singh, V. Dhir, Effectiveness of python libraries in machine learning: A review, Webology (ISSN: 1735-188X) 16 (1) (2019).

[11] S. Saabith, V. Thangarajah, M. Fareez, A review on python libraries and IDEs for data science, South Asian Res. J. Eng. Technol. (2021) 36–53.

[12] M. Lucas, A. Townsend-Teague, M. Neri, S. Poetto, A. Morris, B. Habermann, L. Tichit, Phasik: a python package to identify system states in partially temporal networks, J. Open Source Softw. 8 (91) (2023) 5872, http://dx.doi.org/10.21105/joss.05872.

[13] S. Thivaharan., G. Srivatsun., S. Sarathambekai., A survey on python libraries used for social media content scraping, in: 2020 International Conference on Smart Electronics and Communication, ICOSEC, 2020, pp. 361–366, http://dx.doi.org/10.1109/ICOSEC49089.2020.9215357.

[14] I. Stancin, A. Jović, An overview and comparison of free python libraries for data mining and big data analysis, in: 2019 42nd International Convention on Information and Communication Technology, Electronics and Microelectronics, MIPRO, 2019, pp. 977–982, URL https://api.semanticscholar.org/CorpusID:195883151.

[15] G. Mahalaxmi, A. Donald, T. Aditya, T.A. Srinivas, A short review of python libraries and data science tools, South Asian Res. J. Eng. Technol. 5 (2023) 1–5, http://dx.doi.org/10.36346/sarjet.2023.v05i01.001.

[16] A.H. Sial, S. Yahya, S. Rashdi, D.A.H. Khan, Comparative analysis of data visualization libraries matplotlib and seaborn in python, Int. J. Adv. Trends Comput. Sci. Eng. (2021) URL https://api.semanticscholar.org/CorpusID:241790425.

[17] Y.M. Mohialden, R.W. Kadhim, N.M. Hussien, S.A.K. Hussain, Top python-based deep learning packages: A comprehensive review, Int. J. Pap. Adv. Sci. Rev. 5 (1) (2024) 1–9, http://dx.doi.org/10.47667/ijpasr.v5i1.283, URL http://www.igsspublication.com/index.php/ijpasr/article/view/283.

[18] R.A. Paynter, G.P. Adam, A. Hedden-Gross, C. Twose, C. Voisin, Systematic Review Search Strategy Development Tools: A Practical Guide for Expert Searchers, Agency for Healthcare Research and Quality (US), Rockville, MD, 2025.

[19] F. Beck, M. Burch, S. Diehl, D. Weiskopf, A taxonomy and survey of dynamic graph visualization, Comput. Graph. Forum 36 (1) (2017) 133–159, http://dx.doi.org/10.1111/cgf.12791.

[20] V. Nicosia, J. Tang, C. Mascolo, M. Musolesi, G. Russo, V. Latora, Graph metrics for temporal networks, in: P. Holme, J. Saramäki (Eds.), Temporal Networks, Springer Berlin Heidelberg, Berlin, Heidelberg, 2013, pp. 15–40.

[21] L.R. Pereira, R.J. Lopes, J. Louçã, Community identity in a temporal network: A taxonomy proposal, Ecol. Complex. 45 (2021) 100904, http://dx.doi.org/10.1016/j.ecocom.2020.100904, URL https://www.sciencedirect.com/science/article/pii/S1476945X20301847.

[22] W.H. Thompson, P. Brantefors, P. Fransson, From static to temporal network theory: Applications to functional brain connectivity, Netw Neurosci 1 (2) (2017) 69–99.

[23] E. Aghajani, C. Nagy, O.L. Vega-Márquez, M. Linares-Vásquez, L. Moreno, G. Bavota, M. Lanza, Software documentation issues unveiled, in: 2019 IEEE/ACM 41st International Conference on Software Engineering, ICSE, 2019, pp. 1199–1210, http://dx.doi.org/10.1109/ICSE.2019.00122.

[24] E. Aghajani, C. Nagy, M. Linares-Vásquez, L. Moreno, G. Bavota, M. Lanza, D.C. Shepherd, Software documentation: the practitioners' perspective, in: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, ICSE '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 590–601, http://dx.doi.org/10.1145/3377811.3380405.

[25] Snyk, Snyk python advisor, 2024, (Accessed 03 February 2025) URL https://snyk.io/advisor/python.

[26] P. Rares, Pepy - Python package download stats, 2025, (Accessed 03 February 2025) https://www.pepy.tech.

[27] T.P. Peixoto, The graph-tool python library, Figshare (2014) http://dx.doi.org/10.6084/m9.figshare.1164194, URL http://figshare.com/articles/graph_tool/1164194.

[28] C.L. STAUDT, A. SAZONOVS, H. MEYERHENKE, NetworKit: A tool suite for large-scale complex network analysis, Netw. Sci. 4 (4) (2016) 508–530, http://dx.doi.org/10.1017/nws.2016.20.

[29] A. Mukhopadhyay, GitHub - RecallGraph: A versioning data store for time-variant graph data. — github.com, 2025, (Accessed 03 February 2025), https://github.com/RecallGraph/RecallGraph.

[30] D.A. Bader, D.G. Chavarría-Miranda, STINGER: Spatio-temporal interaction networks and graphs (STING) extensible representation, 2009, URL https://api.semanticscholar.org/CorpusID:18153162.

[31] R. Pfitzner, I. Scholtes, A. Garas, C.J. Tessone, F. Schweitzer, Betweenness preference: Quantifying correlations in the topological dynamics of temporal networks, Phys. Rev. Lett. 110 (2013) 198701, http://dx.doi.org/10.1103/PhysRevLett.110.198701, URL https://link.aps.org/doi/10.1103/PhysRevLett.110.198701.

[32] B.F. Maier, Temporal contact modeling and analysis – tacoma 0.1.19 documentation — rocs.hu-berlin.de, 2018, (Accessed 03 February 2025), http://rocs.hu-berlin.de/~tacoma/index.html.

[33] A.A. Hagberg, D.A. Schult, P.J. Swart, Exploring network structure, dynamics, and function using networkx, in: G. Varoquaux, T. Vaught, J. Millman (Eds.), Proceedings of the 7th Python in Science Conference, Pasadena, CA USA, 2008, pp. 11–15.

[34] N. Aloysio, GitHub - nelsonaloysio/networkx-temporal: Python package to build and manipulate temporal networkx graphs. — github.com, 2025, (Accessed 29 March 2024) https://github.com/nelsonaloysio/networkx-temporal/tree/main.

[35] G. Rossetti, pyup.io bot, E. ter Hoeven, U. Norman, D. Jorquera, H. Dormán, M. Dorner, GiulioRossetti/dynetx: v0.3.2, 2023, http://dx.doi.org/10.5281/zenodo.8009585.

[36] M.L. Ingo Scholtes, GitHub - pathpy/pathpyg: GPU-accelerated next-generation network analytics and graph learning for time series data on complex networks. — github.com, 2025, (Accessed 03 February 2025) https://github.com/pathpy/pathpyG.

[37] R. Shirzadkhani, S. Huang, E. Kooshafar, R. Rabbany, F. Poursafaei, Temporal graph analysis with TGX, in: Proceedings of the 17th ACM International Conference on Web Search and Data Mining, WSDM '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 1086–1089, http://dx.doi.org/10.1145/3616855.3635694.

[38] A. Badie-Modiri, M. Kivelä, Reticula: A temporal network and hypergraph analysis software package, SoftwareX 21 (2023) 101301, http://dx.doi.org/10.1016/j.softx.2022.101301, URL https://www.sciencedirect.com/science/article/pii/S2352711022002199.

[39] L. Oettershagen, P. Mutzel, TGLib: An open-source library for temporal graph analysis, in: 2022 IEEE International Conference on Data Mining Workshops, ICDMW, IEEE Computer Society, Los Alamitos, CA, USA, 2022, pp. 1240–1245, http://dx.doi.org/10.1109/ICDMW58026.2022.00160, URL https://doi.ieeecomputersociety.org/10.1109/ICDMW58026.2022.00160.

[40] F. Poursafaei, S. Huang, K. Pelrine, R. Rabbany, Towards better evaluation for dynamic link prediction, Adv. Neural Inf. Process. Syst. 35 (2022) 32928–32941.

[41] M. Bastian, S. Heymann, M. Jacomy, Gephi: an open source software for exploring and manipulating networks, in: Third International AAAI Conference on Weblogs and Social Media, 2009.

[42] S. Fanton, R. Altawil, I. Ellerbrock, J. Lampa, E. Kosek, P. Fransson, W.H. Thompson, Multiple spatial scale mapping of time-resolved brain network reconfiguration during evoked pain in patients with rheumatoid arthritis, Front. Neurosci. Volume 16 - 2022 (2022) http://dx.doi.org/10.3389/fnins.2022.942136, URL https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2022.942136.

[43] W.H. Thompson, P. Brantefors, P. Fransson, From static to temporal network theory: Applications to functional brain connectivity, Netw. Neurosci. 1 (2) (2017) 69–99, http://dx.doi.org/10.1162/NETN_a_00011.

[44] D. Bartolini, P. Liuzzi, S. Secci, B. Hakiki, M. Scarpino, R. Burali, A. di Palma, T. Toci, A. Grippo, F. Cecchi, A. Frosini, A. Mannini, Temporal network instability and low-frequency overconnectivity underlie disorders of consciousness in severe brain injury, Sci. Rep. 15 (1) (2025) 38835, http://dx.doi.org/10.1038/s41598-025-09233-2.

[45] C. Zhang, J. Hackl, Beyond connectivity: Higher-order network framework for capturing memory-driven mobility dynamics, 2025, arXiv:2507.07727.

[46] J. Hackl, I. Scholtes, L.V. Petrović, V. Perri, L. Verginer, C. Gote, Analysis and visualisation of time series data on networks with pathpy, in: Companion Proceedings of the Web Conference 2021, WWW '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 530–532, http://dx.doi.org/10.1145/3442442.3452052.

[47] M.R. Barnes, V. Nicosia, R.G. Clegg, Measuring social mobility in temporal networks, Sci. Rep. 15 (1) (2025) 5941, http://dx.doi.org/10.1038/s41598-025-89090-1.

[48] A. Kerren, F. Schreiber, Toward the role of interaction in visual analytics, in: Proceedings of the Winter Simulation Conference, WSC '12, Winter Simulation Conference, 2012.